

Java / Spring / Microservices LinkedIn Notes Summary

1. Synchronous vs Asynchronous Programming

Synchronous (SYNC)

- Tasks execute one after another.
- Current thread waits until work finishes.
- Blocking operation.
- Easier to debug and understand.

Asynchronous (ASYNC)

- Tasks can run independently.
- Non-blocking execution.
- Better scalability and responsiveness.

Flow

```
Main Thread -----> Continue other work
      \
      ---> Background Task ---> Callback/Future
```

2. HashMap vs ConcurrentHashMap

HashMap

- Not thread-safe
- Allows one null key and multiple null values

ConcurrentHashMap

- Thread-safe
 - No null keys/values
 - Better for multithreaded systems
-

3. Spring Boot Annotations Cheat Sheet

Core Stereotypes

- @Component
- @Service
- @Repository
- @Controller
- @RestController

REST APIs

- @GetMapping
 - @PostMapping
 - @PutMapping
 - @DeleteMapping
-

4. Spring Framework Overview

Main Concepts

- IoC
- Dependency Injection
- AOP
- Transaction Management

Architecture

```
Application
|
Spring Framework
  ■■■ Web
  ■■■ AOP
  ■■■ Data Access
  ■■■ Core Container
```

5. Strategy Pattern

Benefits

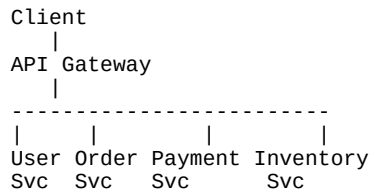
- Cleaner code
 - Better testing
 - Easy extension
 - Follows Open/Closed Principle
-

6. Microservices Design Patterns

Important Patterns

- Saga Pattern
- Circuit Breaker
- Retry with Backoff
- API Gateway
- Event Driven Architecture
- Bulkhead Pattern
- Service Discovery

Flow



7. Java Records

Example

```
record User(String name, int age) {}
```

Benefits

- Immutable
 - Less boilerplate
 - Auto-generated methods
-

8. Java Collections Framework

Main Types

- List
- Set
- Queue
- Map

Collection Hierarchy

```
Collection
  ■■■ List
  ■■■ Set
  ■■■ Queue

Map
  ■■■ HashMap
  ■■■ TreeMap
  ■■■ ConcurrentHashMap
```

9. Java 8 Features

Features

- Lambda Expressions
- Stream API
- Functional Interfaces
- Optional
- Date & Time API

Lambda Example

```
list.forEach(name -> System.out.println(name));
```

Final Interview Preparation Notes

Focus Areas

1. Collections

2. Java 8
3. Spring Boot
4. Microservices
5. Concurrency
6. Design Patterns

Recommended Order

```
graph TD; A[Java Basics] --> B[Collections]; B --> C[Java 8]; C --> D[Spring Boot]; D --> E[Microservices];
```

Java Basics
↓
Collections
↓
Java 8
↓
Spring Boot
↓
Microservices